Mikhail Razakov

Compiler Backend / Code Generation Engineer

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop experimental compiler-backend components in Rust, C++/LLVM and C#/.NET. My Rust project covers SSA/CFG validation, register allocation, spills, phi lowering and SysV x86-64 emission. An independent verifier checks assignments, and native execution is compared with a reference IR interpreter. At MCST I worked on experimental LLVM LICM; I now work on symbolic computation for SharpChecker at ISP RAS.

TECHNICAL SKILLS

Backend / Codegen: Rust, iced-x86, SSA/CFG, dominance, edge-specific liveness, linear scan, seeded simulated annealing, spills, phi lowering, SysV x86-64

Compiler / Runtime: C++23, LLVM 22/IR, APInt, IR transformations; C#, .NET, CIL/CLR, interpreter/backend execution

Correctness / Tooling: Independent assignment verifier, reference interpreter, differential testing, Cargo tests, Clippy, Python, Git, Linux, CMake

EXPERIENCE

ISP RAS · SharpChecker · October 2026 — present

• Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship · July — August 2026

• Worked on experimental LLVM LICM, checking instruction-hoisting legality against loop structure, memory access and side effects; compared before/after behavior using Python.

ENGINEERING PROJECTS

x86-64 Compiler Backend & Register Allocation

Built a backend for restricted i64 IR: SSA/CFG, liveness, linear-scan and seeded simulated-annealing allocation, spills, phi lowering and SysV x86-64 emission. An independent verifier rejects register/stack-slot conflicts; phi lowering handles parallel-copy cycles. Native execution runs in a separate process and is compared with an IR interpreter. Passed 42/42 local debug tests, cargo fmt and strict Clippy on commit dd0693c; local release tests were blocked by disk quota during compilation.

LLVM Interprocedural Global IV Optimization

An LLVM pass analyzes interprocedural effects and affine evolution of selected integer-global accesses, then checks legality and conservatively rewrites IR. Passed 29/29 regression cases on LLVM 22.1.8, including unsupported cases where the transformation is rejected. This is not an LLVM target backend.

UniversalToolchain / Wist2

Developing modular infrastructure with Bytecode/AIR, interpreter and CIL backends. LanguagePlan records selected execution routes; tests check interpreter/CIL parity for supported behavior. The repository manifest records 1,324 expected passing tests, not a fresh full-suite run during this CV audit.

EDUCATION

HSE University — Software Engineering

RECOGNITION

2026 · LangDev — UniversalToolchain: Presented the compiler infrastructure architecture on 9 October 2026. — Talk recording

2025–2026 · MEPhI Junior: First-degree diplomas in the Information Technologies section in 2025 and 2026.

2026 · MCST LLVM/C++ selection: According to organizer correspondence: 5.0/5.0 and 104/104 official tests among 49 submitted solutions.